

PLAN.md — Rime Hackathon: Voice-Native Interruption & Recovery Agent

Status: draft for team + AI coding agents · Repo: `rime_hackathon` (Next.js frontend already exists)

0. Why this document exists

The problem statement changed. This doc re-reads the new PDF (`Rime Hackathon Challenge – Build a Voice-Native Product`), maps it onto the frontend you already built, and defines:

What hard voice problem we are actually proving (chosen deliberately, not left ambiguous).

What the frontend still needs to build/change.

A basic Python backend flow (STT → LLM → Rime TTS) including where "AI" and "transformer" pieces actually live.

A **contract** between frontend and backend — the single source of truth both sides must code against — plus the process for changing that contract without breaking the other side.

A merge checklist so two teams (or two AI agents) working in parallel don't produce incompatible code.

Any AI agent (Claude, Cursor, Copilot, etc.) working on this repo should read this file plus `AGENTS.md` before writing code. If the two ever conflict, `AGENTS.md`'s hard rules win for frontend-internal conventions; this file wins for cross-boundary (frontend↔backend) contract decisions.

1. Chosen hard voice problem (this is our PS)

The PDF requires picking **one** hard voice problem and defining its acceptance test before the demo. Based on what's already built (barge-in flush, WS `user_interrupt` event, session replay with latency badges), we target:

Interruption and recovery, extended into **conversation continuity during tool/LLM work**.

Target user & situation: someone talking to a voice assistant that sometimes needs to "look something up" or "run a tool call" (e.g. checking an order, pulling a fact, doing a calculation) before it can answer. The user should be able to interrupt at any point — while the assistant is speaking, or while it's silently waiting on a slow tool/LLM call — change their request, and get a response that reflects only what they actually said and heard, never a stale answer to the old request.

Why voice is essential: if you strip TTS out, the "interrupt mid-speech" problem doesn't exist — there's nothing playing to interrupt. The entire challenge (stopping audio instantly, fencing stale results, keeping transcript/state consistent with what was *heard*, not just what was *generated*) only exists because output is spoken, not read. That satisfies the "problem and necessity of voice" judging criterion directly.

Acceptance test (define before demo, per PDF requirement):

Setup: inject a fixed artificial delay (e.g. 3s) into one backend tool/LLM call.

Normal path: user asks a question requiring the slow tool call → assistant speaks the correct final answer after the delay.

Stress case: while the assistant is mid-sentence (or mid-wait, before any audio has started), the user barges in with a *different* request that changes the answer (e.g. "actually, make that for next Tuesday instead").

Pass criteria (all must hold, and are what we measure/screen-record):

Rime audio output stops within a measured threshold (target: **<150ms** from `user_interrupt` receipt to local playback silence) — measured client-side, not assumed.

The stale in-flight LLM/tool result for the old request is discarded server-side and **never spoken** (verified via `generationId`/`turnId` fencing — see §4).

The new request is processed using the updated instruction, and the final spoken response matches it.

The transcript/session-replay view (`/live-chat/[sessionId]`) shows exactly what the user heard — no "ghost" turns for audio that was cut off before completion.

Measurement artifact: a script/fixture in `backend/tests/interrupt_stress_test.py` that drives this scenario headlessly against the WS server and asserts on the event sequence and timestamps, plus one recorded clip for the README/demo.

This is the acceptance test that goes into `RIME_EVIDENCE.md` per the submission requirements.

2. Frontend plan (builds on existing repo structure)

Existing structure (from repo): Next.js 16 App Router, TypeScript, Tailwind v4, Better Auth, monochrome design system, contract-first `src/contracts/`, `api.<feature>.<op>()` client, `/dev/api` contract explorer, WS protocol already sketched in `AGENTS.md`.

Keep as-is (already fits the new PS):

`/sign-in`, `/dashboard`, monochrome design system, mock-badge convention, `getSession(await headers())` gating.

`/live-chat` — real-time voice chat, barge-in flush on `user_interrupt`.

`/live-chat/[sessionId]` — turn transcript replay with latency badges (this becomes your *evidence viewer* for the acceptance test).

`/dev/api` contract explorer — keep this; it's how backend agents can self-check without reading frontend code.

Add / change for this PS:

Item	Why
<code>src/contracts/interrupt.ts</code>	New contract: request/response + WS event shapes for the fencing mechanism (<code>generationId</code> , <code>cancel_reason</code>). See §4.2.
Latency instrumentation on <code>/live-chat</code>	Must capture and display: time-to-first-audio-byte, time-to-silence-after-interrupt, per turn. Feed into <code>state_sync</code> .
"Stress test" trigger in dev tools (<code>/dev/api</code>) or a hidden <code>?stress=1</code> flag)	Lets judges/team reproduce the deliberate failure case live without editing code — required by "demo clarity" + "evidence and reproducibility" criteria.
Provider-status indicator (already partially covered by <code>provider_status</code> WS event)	Must be visibly rendered per turn — PDF requires "which speech provider is active" to be shown in the demo.
<code>RIME_EVIDENCE.md</code> renderer (optional)	A simple <code>/dev/evidence</code> page that fetches and displays the latest stress-test run result — nice-to-have for demo clarity, not required.

Do not add: any database, any direct LLM/Rime calls from the frontend, any raw `fetch()` outside `api-client` (unchanged hard rules from `AGENTS.md`).

2.5 Locked tech stack (authoritative — read this before writing any backend or frontend code)

This supersedes any generic "FastAPI + raw WebSocket" assumption elsewhere in this doc or in `AGENTS.md`. If an AI agent is about to scaffold a plain `websockets`/`fastapi.WebSocket` server or a hand-rolled browser `WebSocket` client for audio, **stop** — that is not this stack. The table below is locked for the hackathon; do not substitute providers or swap LiveKit for a custom transport to "simplify," since interruption handling, VAD, and turn-taking are exactly what LiveKit Agents already solves.

Layer	Recommended / allowed stack	Why
Spoken output (TTS)	Rime TTS — <code>coda</code> / <code>mistv3</code> models, via <code>livekit-plugins-rime</code> (preferred) or Rime's WebSocket API directly	Mandatory core output. WebSocket streaming targets sub-200ms TTFB (time-to-first-audio-byte) with word-level aligned timestamps.
Realtime transport & full-duplex	LiveKit Agents — <code>livekit-agents</code> Python SDK, WebRTC	Officially recommended by Rime. Handles bidirectional audio, low-latency streaming, and turn-taking out of the box — this is the layer that replaces a hand-rolled WebSocket protocol.
Speech-to-text (STT)	Deepgram Nova-2 / Cartesia / Groq Whisper-v3	Streaming, sub-150ms endpointing, intermediate (partial) transcripts.
Orchestration & reasoning (LLM)	Fast-inference LLM — Groq Llama-3.3-70B / Cerebras / OpenAI GPT-4o-mini	First-token latency under ~150ms is critical for perceived response time — this is the "transformer" piece; call it like any hosted API, don't self-host.
Voice activity detection (VAD)	Silero VAD , LiveKit-integrated	Near-instant interruption detection and speech-boundary segmentation — this is what fires <code>user_interrupt</code> -equivalent events, so it directly powers the acceptance test in §1.
Frontend / interface	Next.js 15 (React 19) , Tailwind CSS, LiveKit Components Web (<code>@livekit/components-react</code> , <code>livekit-client</code>)	Audio visualizer, live audio-latency waterfall, dynamic conversation/debug inspector. Replaces raw <code>WebSocket</code> audio streaming in the browser.
Evaluation & benchmarking	Python, PyDub, SciPy, JiWER, Playwright	Repeatable test scripts for time-to-first-audio (TTFA) and interruption-recovery metrics; JiWER for transcript word-error-rate scoring, Playwright for driving the browser end of the stress test.

2.5.1 What this changes vs. the existing repo

Your frontend currently implements a **custom raw WebSocket protocol** (per `AGENTS.md`: `start_session`, `audio_chunk` as base64 JSON, `tts_audio_chunk` as base64 JSON, etc.). Moving to LiveKit is a real migration, not an additive change:

Microphone audio is no longer sent as base64 `audio_chunk` messages — it becomes a published **LiveKit audio track** (`room.localParticipant.setMicrophoneEnabled(true)`), transported over WebRTC.

Rime TTS audio is no longer sent as base64 `tts_audio_chunk` messages — the backend agent publishes it as an **outbound audio track** in the same LiveKit room; the frontend subscribes to it like any other participant's track and it just plays.

Everything that isn't audio (transcripts, `user_interrupt` signal, `provider_status`, `state_sync`, `error`) still travels as JSON, but over **LiveKit's data channel** (`room.localParticipant.publishData(...)` / the `DataReceived` event) instead of a raw `ws.send`. The event *shapes* defined in §4.2 stay the same — only the transport underneath them changes from "your own WS server" to "LiveKit's data channel."

`NEXT_PUBLIC_WS_BASE_URL` is replaced by a LiveKit server URL + a short-lived room access token the backend mints per session (`POST /sessions` → `{ livekitUrl, token, roomName }`); the frontend connects with `livekit-client`'s `Room.connect(livekitUrl, token)` instead of `new WebSocket(...)`.

Update `AGENTS.md` rule 7 ("send `user_interrupt` event to WebSocket") to say "send `user_interrupt` as a LiveKit data message" — flag this to whoever owns that file so it doesn't silently contradict this plan.

If there's no time to do a full LiveKit migration mid-hackathon, the fallback is to keep the existing raw-WebSocket contract in §4.2 exactly as-is and treat LiveKit as a "if time permits" upgrade — but then Rime's official integration path and the

"officially recommended" judging goodwill are lost, and VAD/turn-taking has to be hand-rolled instead of coming from Silero for free. Decide this explicitly as a team, don't let it default silently.

3. Backend plan (Python) — basic flow

This section is written so a team member with limited AI/backend background can hand it directly to an AI coding agent and get a working service.

3.1 What "AI" pieces are actually needed, in plain terms

Four components per the locked stack (§2.5), chained together per turn inside a **LiveKit Agent** process. None of them need to be a custom-trained "transformer" — all are hosted APIs/plugins you call. You are **not** training or hosting a model yourself.

VAD — Silero, via LiveKit's built-in integration. Detects when the user starts/stops talking and when they barge in over the agent's own speech. This is what triggers interruption handling in §3.3 — you mostly configure it, you don't hand-write speech-boundary detection.

Speech-to-text (STT) — Deepgram Nova-2 (primary), Cartesia or Groq Whisper-v3 as alternates. Turns mic audio into text, streaming (partial + final transcripts), sub-150ms endpointing. Used via `livekit-plugins-deepgram` inside the Agent pipeline.

LLM (the "brain") — Groq Llama-3.3-70B, Cerebras, or OpenAI GPT-4o-mini, whichever measures the lowest first-token latency in your environment. Takes the transcript + conversation state and decides what to say / which tool to call. This is the actual "transformer"; call it like any hosted API, streaming tokens back as they're generated so TTS can start speaking before the full response is done. Target: first-token latency under ~150ms.

Must support **cancellation**: when the user interrupts, stop reading further streamed tokens and discard what's left. LiveKit Agents' pipeline gives you a cancellation hook for this — use it rather than rolling your own.

Rime TTS — `coda` or `mistv3` model, via `livekit-plugins-rime`. Takes LLM text (ideally sentence-by-sentence as it streams in) and returns audio, published as a LiveKit audio track. This is the **required, primary spoken output** — non-negotiable per the PDF's eligibility rules. Target sub-200ms TTFB (time-to-first-audio-byte), and use Rime's word-level aligned timestamps to know exactly what the user actually heard before an interrupt cut playback off (needed for `deliveredText` in `state_sync`, §4.2).

The "hard voice engineering" is not any one of these four calls — it's the **orchestration layer** around them (LiveKit Agents' pipeline + your own turn-fencing logic) that handles interruption, cancellation, and state fencing. That's the part worth the most engineering effort and judging points.

3.2 Suggested backend structure


```
backend/
app/
  main.py          # FastAPI app: NON-realtime HTTP only — /health, /voices, /sessions (token minting), /evidence
  agent.py         # LiveKit Agent entrypoint (livekit-agents worker) — the realtime process
  turn_manager.py  # Core interruption/fencing logic (see 3.3), invoked from agent.py's callbacks
providers/
  stt_deepgram.py  # livekit-plugins-deepgram config + fallback wiring
  llm_client.py    # Groq / Cerebras / OpenAI client, streaming + cancellable
  tts_rime.py      # livekit-plugins-rime config: model (coda/mistv3), voice, language
  vad_silero.py    # Silero VAD config (thresholds, min-speech/min-silence durations)
contracts/
  data_events.py   # Pydantic models for LiveKit data-channel messages, mirroring src/contracts/*.ts EXACTLY
  __init__.py
config.py          # Env var loading, provider selection, fallback logic, Rime catalog validation
tests/
  interrupt_stress_test.py # The acceptance-test fixture from §1 — drives a real LiveKit room + Playwright browser client
requirements.txt   # livekit-agents, livekit-plugins-rime, livekit-plugins-deepgram, livekit-plugins-silero, fastapi, pydub,
scipy, jiwer, playwright
.env.example
README.md          # Setup, architecture, exact Rime model/voice/endpoint/transport used
```

Note the split: `main.py` is a normal FastAPI app for the small number of non-realtime HTTP calls (minting LiveKit room tokens, exposing the voice catalog, serving evidence). `agent.py` is a separate long-running **LiveKit Agents worker process** (started via `python agent.py dev` / deployed as a LiveKit Agent) that actually joins rooms and runs the STT→LLM→TTS pipeline — it is not a route inside the FastAPI app. Don't collapse these into one process; LiveKit's worker model expects the agent to be its own dispatchable process per room.

3.3 Turn manager / interruption logic (the actual hard part)

This logic lives inside `agent.py`, wired into LiveKit Agents' pipeline callbacks (e.g. its user-speech-committed / interruption hooks) rather than a hand-rolled WS message loop — LiveKit tells you *when* VAD detected a barge-in, `TurnManager` decides *what to do about it* (which in-flight work to cancel, what counts as stale). Core idea: every assistant "turn" (one STT-final → LLM → TTS cycle) gets a monotonically increasing `generationId`. Nothing gets spoken or applied to session state unless its `generationId` is still the current one.

□
python

turn_manager.py (sketch — not production code, illustrates the pattern)

```
class TurnManager:
    def __init__(self):
        self.current_generation = 0
        self.active_task: asyncio.Task | None = None

    async def start_turn(self, user_text: str, session):
        self.current_generation += 1
        my_gen = self.current_generation
        if self.active_task and not self.active_task.done():
            self.active_task.cancel() # cancel stale LLM/tool call
        self.active_task = asyncio.create_task(
            self._run_turn(user_text, my_gen, session)
        )

    async def _run_turn(self, user_text, my_gen, session):
        async for token in llm_client.stream(user_text):
            if my_gen != self.current_generation:
                return # fenced: a newer turn started
            sentence_buffer += token
            if sentence_boundary(sentence_buffer):
                async for audio_chunk in tts_rime_client.stream(sentence_buffer):
                    if my_gen != self.current_generation:
                        return # fenced mid-speech: stop pushing frames to the outbound track
                    await session.audio_track.push_frame(audio_chunk) # published as a real LiveKit audio track, not a WS message
                sentence_buffer = ""

    def handle_interrupt(self, session):
        self.current_generation += 1 # invalidates everything in flight
        if self.active_task:
            self.active_task.cancel()
        session.audio_track.clear_queue() # stop whatever's already buffered for playback
        # tell frontend what was actually delivered, over the LiveKit data channel:
        session.publish_data({"type": "state_sync", "interrupted": True, ...})
```

Key properties this must satisfy (map directly to acceptance-test pass criteria in §1):

Cancelling `active_task` must actually stop the LLM stream and Rime stream (close the underlying HTTP/WS connections), not just stop reading them — otherwise you're still paying for/generating audio nobody hears.

The generation check must happen **inside the streaming loop**, not just once at the start, since a slow tool call can be interrupted mid-wait before any tokens exist.

`state_sync` must be sent immediately on interrupt (before the new turn's audio starts) so the frontend can flush its local playback buffer without waiting on the network round-trip for new audio.

3.4 Fallback / provider visibility (required by PDF eligibility rules)

Backend must track which speech provider actually served the last utterance and emit it via `provider_status`. Rime is the default; only fall back (and disclose it) if Rime's call fails.

Never let a fallback happen silently — the PDF explicitly disqualifies incidental/hidden fallback.

4. The contract: what frontend and backend must agree on

This is the part that prevents merge conflicts. Both sides code against this section, not against each other's source.

4.1 How contracts are formed (process, not just schema)

Single source, two representations. The contract is authored once in prose/JSON-schema form (this section). Frontend encodes it as Zod/TS types in `src/contracts/<feature>.ts`. Backend encodes it as Pydantic models in

`app/contracts/ws_events.py`. These must be structurally identical — same field names, same types, same optionality.

Naming convention across the wire: `camelCase`, always. JSON payloads use `camelCase` on both sides (e.g. `sessionId`, `turnId`, `audioData`) even though Python idiomatically prefers `snake_case` internally — the backend converts at the serialization boundary (Pydantic `alias`/`by_alias=True`), it never leaks `snake_case` into a WS message.

Any contract change is a PR that touches three things at once: the prose spec here, `src/contracts/<feature>.ts`, and `app/contracts/ws_events.py`. A PR that changes only one of the three should be rejected in review — that's the #1 source of integration breakage.

Mock parity is mandatory. Every frontend contract has a deterministic `mock()`. When the real contract changes, the mock must change in the same PR, or `/dev/api` and `NEXT_PUBLIC_USE MOCKS=true` mode silently drift from reality.

Versioning: if a breaking change is unavoidable mid-hackathon, bump a `protocolVersion` field in `start_session` and have the backend reject/negotiate rather than silently misinterpret old messages.

4.2 Realtime contract under LiveKit (supersedes a raw WebSocket protocol — read §2.5.1 first)

Two separate channels now carry what used to be one WS connection: real **audio tracks** (mic in, Rime TTS out — never JSON, never base64) and the **LiveKit data channel** (everything else, still JSON, same shapes as before). Session start is no longer a WS message — it's an HTTP call that mints a room + token, then the client joins with `livekit-client`.

Session start (HTTP, replaces `start_session`):

`POST /sessions` → request `{ voiceId?: string, protocolVersion: 1 }` → response `{ sessionId, roomName, livekitUrl, token, protocolVersion: 1 }`. Frontend then calls `Room.connect(livekitUrl, token)` and publishes its microphone track — this join is the equivalent of the old `start_session` + implicit `audio_chunk` stream.

End of session: `room.disconnect()` client-side; backend agent detects the participant-left LiveKit event and tears down state (replaces `end_session`).

Audio (tracks, not messages):

What	Direction	Notes
Microphone	client → room (published LiveKit/WebRTC handles codec/format negotiation — do not hand-roll track)	<code>audio/webm;codecs=opus</code> chunking.
Rime TTS speech	backend agent → room (published track)	Backend pushes PCM frames from <code>livekit-plugins-rime</code> into an outbound track; frontend subscribes automatically once the agent joins the room.

Data-channel messages (JSON, same field names/casing rules as before, just sent via `publishData`/received via `RoomEvent.DataReceived` instead of `ws.send`/`onmessage`):

Event	Shape	Notes
<code>text_message</code>	<code>{ type, content }</code>	Used by TTS Studio / text-only test paths (no mic track needed for this one). In practice this is usually <i>derived</i> from LiveKit/Silero VAD's own barge-in detection inside <code>agent.py</code> , and re-broadcast as this event for the frontend's UI/logging — but the frontend may still publish it explicitly (e.g. a manual "stop talking" button), and the backend must treat either source identically.
<code>user_interrupt</code>	<code>{ type, timestamp: epoch_ms, reason? }</code>	
<code>provider_status</code>	<code>{ type, provider: 'rime' 'fallback' }</code>	Sent at session start and on any provider change.
<code>partial_transcript</code>	<code>{ type, role, text, turnId? }</code>	STT interim results, not final.
<code>final_transcript</code>	<code>{ type, role, text, turnId, latencyMs? }</code>	<code>latencyMs</code> = end-of-user-speech → this event, for the perceived-response-time story.
<code>state_sync</code>	<code>{ type, turnId, deliveredText, interrupted: boolean }</code>	Sent right after any interrupt is processed; frontend uses <code>deliveredText</code> (not the full LLM output, and derived from Rime's word-level timestamps up to the cutoff) as the transcript-of-record for what was actually heard.
<code>error</code>	<code>{ type, message, code? }</code>	<code>code</code> enum: <code>stt_error</code> , <code>llm_error</code> , <code>tts_error</code> , <code>provider_fallback</code> , <code>session_not_found</code> , <code>internal_error</code> . Frontend must handle all six explicitly, not just show a generic error.

Field-level gotchas that cause silent bugs if missed:

`timestamp` / `latencyMs` are **epoch milliseconds, integers**, never ISO strings — pick one format and never mix; mixing is the single most common contract bug.

`turnId` is a string, generated by the **backend**, unique per generation — frontend never generates its own `turnId`.

Do not reintroduce `audio_chunk` / `tts_audio_chunk` base64 messages "just to be safe" — if audio is flowing over the data channel at all, the LiveKit migration wasn't actually done; audio only ever moves as a track.

There is no `isLast` flag anymore for audio (tracks don't end mid-stream the way a chunked message list does) — "turn is done speaking" is instead signalled by the `final_transcript` / `state_sync` data message plus the track simply going quiet; frontend should use a short trailing-silence timeout on the audio visualizer rather than waiting for an explicit end marker.

4.3 HTTP contract (non-realtime)

Endpoint	Method	Purpose
<code>/health</code>	GET	Used by frontend to show backend-connected vs mock-mode status.
<code>/sessions</code>	POST	New, LiveKit-specific. Mints a <code>roomName</code> + short-lived LiveKit access <code>token</code> ; see §4.2. Requires the auth header below.
<code>/voices</code>	GET	Passthrough of Rime's current voice/model/language catalog — frontend TTS Studio must not hardcode a stale voice list per PDF's "use current catalog at submission time" rule.
<code>/sessions/{id}/transcript</code>	GET	Backs the <code>/live-chat/{sessionId}</code> replay page — must include <code>deliveredText</code> , not raw LLM output, for turns that were interrupted.

Endpoint	Method	Purpose
<code>/evidence/latest</code>	GET	Returns the last <code>interrupt_stress_test.py</code> run result as JSON, for the optional <code>/dev/evidence</code> page.

4.4 Auth handoff

Frontend uses Better Auth (Google sign-in) and owns the session cookie. Backend does **not** re-implement OAuth. Simplest correct approach for a hackathon: frontend sends the Better Auth session token as a header (`Authorization: Bearer <token>`) on the `POST /sessions` call; backend validates it against a small frontend-exposed verification route (or just trusts it and treats the resulting `roomName` as opaque, if judges' scoring doesn't require per-user auth on the backend — confirm with team, this is a place to cut scope if time-constrained). Once the LiveKit `token` is issued, LiveKit itself handles authenticating the realtime connection — the Better Auth token's only job is gating who can call `/sessions` in the first place.

5. Backend instructions for the team / AI agents (hard rules, mirrors

`AGENTS.md`)

Give this list verbatim to whoever/whatever builds the backend:

Rime is the only TTS path in the judged flow. Any other TTS is fallback-only and must emit `provider_status: 'fallback'` when used — never silent.

Never commit API keys. Rime key, LLM key, STT key all live in backend-only env vars (`.env`, gitignored), never in `.env.example` beyond a placeholder, never in frontend code, never in logs that get committed.

Every WS event you emit must match §4.2 exactly — field names, types, casing. Before writing a handler, check `src/contracts/*.ts` in the frontend repo (or ask for it) rather than inventing a shape.

Cancellation must be real, not cosmetic. `user_interrupt` must actually close/cancel the in-flight LLM and Rime streaming calls, not just stop relaying their output to the client — otherwise you're not solving the stated hard problem, you're hiding it.

Log every turn's timing (STT-final timestamp, LLM-first-token timestamp, TTS-first-byte timestamp, interrupt-received timestamp, playback-stop-confirmed timestamp if the frontend reports it back). This is what `RIME_EVIDENCE.md` and the stress-test fixture are measured from — don't bolt this on at the end.

Use the current Rime catalog at call time, not a hardcoded model/voice/language list — fetch `/voices` (or Rime's live catalog endpoint) and validate the configured model/voice/language combo against it at startup; fail loudly if invalid rather than silently falling back.

Write the stress-test fixture (`tests/interrupt_stress_test.py`) before declaring the feature done. It should be runnable with one command and print pass/fail against the four criteria in §1.

Don't touch frontend code. If a contract needs to change, edit the prose spec in this file + both contract files in the same PR (§4.1), and flag it to whoever owns frontend — don't unilaterally reshape a WS event to make backend code simpler.

6. Merge checklist (run through this before connecting frontend to real backend)

- ☐ FastAPI HTTP app (`main.py`) running on `localhost:8000`, matches `NEXT_PUBLIC_API_BASE_URL` in frontend `.env.local`.
- ☐ LiveKit server URL configured on both sides (`LIVEKIT_URL` backend env, returned to frontend via `POST /sessions` — frontend should never hardcode it separately).
- ☐ LiveKit Agent worker (`agent.py`) actually running and auto-dispatching into rooms created by `/sessions` — a room with no agent in it is the #1 "why is nothing happening" bug with this stack.

- ☐ `NEXT_PUBLIC_USE MOCKS=false` toggled, and every page still works (no page was silently depending on mock-only behavior).
- ☐ Frontend confirmed to be using `livekit-client`/LiveKit Components Web for the mic/audio path, not a leftover raw `WebSocket` from the pre-migration code.
- ☐ All six `error.code` values handled in frontend UI, not just logged.
- ☐ `turnId` format from backend actually round-trips correctly into the `/live-chat/[sessionId]` replay page.
- ☐ Timestamps confirmed epoch-ms on both sides (grep both codebases for any ISO-string date handling on data-channel fields).
- ☐ LiveKit room/token config (CORS equivalent — API key/secret, allowed origins) includes the frontend's actual dev/deploy origin.
- ☐ Run `tests/interrupt_stress_test.py` (Playwright driving the real browser client + real LiveKit room) against the real (not mocked) backend and confirm it passes the §1 criteria end-to-end.
- ☐ Run a JiWER pass comparing STT transcripts against a small labeled fixture set, so "pronunciation and delivery" claims in the README are measured, not asserted.
- ☐ Record the 4-5 min demo only after this checklist passes — don't demo against mocks.

7. Suggested order of work

Backend skeleton: `POST /sessions` minting a real LiveKit room + token, and a minimal `agent.py` that joins the room and just echoes the mic track back (no STT/LLM/TTS yet) — prove the LiveKit plumbing works end-to-end before adding AI.

Frontend: connect via `livekit-client`/LiveKit Components Web against that skeleton, confirm audio round-trips.

Wire in STT (Deepgram) → LLM (Groq/Cerebras/GPT-4o-mini) → Rime TTS happy path in `agent.py` (no interruption handling yet).

Add `TurnManager` generation-fencing (§3.3) hooked into LiveKit's VAD/interruption callbacks — this is the graded "hard voice engineering" piece, budget the most time here.

Write `interrupt_stress_test.py` (Playwright + real LiveKit room) and get it passing.

Frontend: latency instrumentation + stress-test trigger + provider-status display polish.

`RIME_EVIDENCE.md` + README with exact model/voice/endpoint/audio-format/transport used (`livekit-plugins-rime`, model = `coda` or `mistv3`, etc.).

Record demo, run merge checklist one final time against the exact demo build.